Distributed Systems
2026WS

universität innsbruck

703088 (2026WS) - PS Distributed Systems

Distributed and Parallel Systems Group
University of Innsbruck
Instructors: Juan Aznar Poveda, Zahra Najafabadi Samani, and Marlon Etheredge
Author(s): Marlon Etheredge
In case of questions regarding this task, please contact the author (marlon.etheredge@uibk.ac.at).

EX02 - Shared State

Introduction

In this exercise, you will explore how state is shared between nodes in a distributed system and how different approaches affect consistency and performance.

We distinguish three types of state:

- **Private state:** State owned by one node, invisible to others. No synchronization is needed.
- **Replicated state:** Each node maintains its own copy. Nodes must exchange and merge state to maintain consistency.
- **Single-copy state:** Nodes access one authoritative copy in a shared store. The store coordinates concurrent access using atomic operations or synchronization mechanisms such as locks.

You will implement a distributed counter using two different approaches and compare their behavior.

In this exercise, use a programming language of your choice. Implement the distributed system as separate programs communicating through a network, preferably executed on separate computers.

Task 1 - Replicated counter

Consider two nodes, A and B, which maintain their own counter values. Both nodes can increment their local counter and exchange their values through messages.

- **1.1).** Implement a program in which each node sends its counter to the other node. When receiving a counter value, the node updates its own counter using:

  ```
  counter = max(local_counter, received_counter)
  ```

  Demonstrate that the two counters eventually become equal if increments stop and synchronization messages are delivered.

- **1.2).** Consider the following situation:
  - Initially, both counters are zero.
  - A increments its counter once.
  - At the same time, B increments its counter once.
  - Both nodes exchange their counter values.

  What is the resulting counter value? What should it be?

  Explain why the two nodes agreeing on a value does not necessarily mean that the value is correct.

- **1.3).** Consider what happens if:
  - A synchronization message is lost.
  - The same message is received twice.
  - An older message arrives after a newer one.

  Which situations cause problems for your implementation? Explain briefly.

- **1.4).** Modify your protocol so that increments performed independently by A and B are not lost.

Hint: Instead of maintaining a single integer, each node could maintain two integers: one for increments performed by A and one for increments performed by B. How could these values be merged?

Show that your improved solution produces the correct total when both nodes increment concurrently and subsequently exchange their state.

Task 2 - Single-copy counter

Instead of maintaining two replicas, the nodes now share one counter stored in a common data store. For this purpose, freely choose one, such as etcd, NATS, Memcached, or Redis.

Both nodes can read and increment the counter. The data store must provide a mechanism to synchronize concurrent updates, such as atomic operations or locks.

- **2.1).** Implement an increment operation that safely updates the shared counter.

  Run increments from both nodes and verify that the resulting counter value is correct.

  Explain why this works without requiring the two application nodes to exchange synchronization messages.

- **2.2).** Consider what happens when both nodes attempt to increment the counter simultaneously.

  What could go wrong if both nodes read the current value, increment it locally, and then write the result back?

  Explain how your chosen data store prevents this problem.

  Investigate what happens when one node attempts to update the counter while another update is in progress. Depending on your implementation, this may involve holding a lock or performing concurrent atomic operations.

  What happens to the second update? Does it wait, fail, or complete immediately?

  Explain the observed behavior and how the data store ensures consistency.

Task 3 - Comparison

You have implemented two approaches to shared state. Compare them by answering the following questions.

- **3.1).** Measure the time required to perform 100 increments using each implementation.

  For the replicated approach, distinguish the time needed to update local state from the time needed to synchronize both replicas.

  For the single-copy approach, measure the time required for an update to the shared store. Investigate whether concurrent updates affect the execution time, for example due to locking or other synchronization mechanisms.

  Report your measurements and briefly explain the differences.

- **3.2).** Answer the following questions:

  1. Where does synchronization happen in each implementation?
  2. What happens if the two nodes cannot communicate temporarily?
  3. What happens if the shared data store is unavailable?
  4. Which approach would you choose if increments must be immediately visible to both nodes? Why?

Submission

Submit your source code and a short report containing your answers, observations, and measurements.

You may assume that each increment request is executed exactly once at its receiving node, that nodes do not crash, and that communication eventually resumes after temporary interruptions.

The submission should follow the following structure and naming convention:

```
|-- exc0X_report_csXXXXXX.pdf
`-- exc0X_source_csXXXXXX.zip
    |-- README.md
    |-- sample_file_a.py
```

```
    |-- sample_file_b.py
    `-- sample_file_c.py
```

Deadline

29.10.2026 12:00h

Presentation

30.10.2026